University of Minnesota custom Azure course

Session 1 Participant Workbook

Azure Storage Architecture for Multi-Petabyte Portfolios

Azure Storage, Data Protection, and Resilience at Scale

Session: Day 1, Session 1, four hours

Use: Checkpoint discussion, design exercise or lab debrief, discussion prompts, knowledge check

Keep: Your worksheets are the input to the Session 6 roadmap capstone

All scenarios use synthetic data modeled on a large public research university. No real University of Minnesota data is used.

Contents

1. Session map 3
1.1 What this session covers 3
1.2 Have ready 3
1.3 Materials you will receive 3
2. Block 1 checkpoint: three questions before data lands 4
3. Design exercise: mapping three university data classes to Azure storage 5
3.1 Steps 5
3.2 Worksheet: account and namespace decisions 5
3.3 Worksheet: tiers, lifecycle rules, and tags 6
3.4 Worksheet: redundancy, sharding, and cost 7
3.5 What you present 8
3.6 How your design will be critiqued 8
4. Discussion prompts 9
5. Knowledge check 10
6. Takeaways and the capstone 11
7. Notes 11

1. Session map

Day 1, Session 1: Azure Storage Architecture for Multi-Petabyte Portfolios. Four hours including a fifteen-minute break.

Block	Minutes	Activity
Block 1	60	Storage account architecture and namespace decisions
Block 2	50	Access tiers, lifecycle management, and the tiering demonstration
Break	15	
Block 3	50	File, NAS, and HPC storage options; scale targets and sharding
Block 4	65	Design exercise and group presentations

1.1 What this session covers

The university's storage portfolio spans several petabytes, most of it research data that ranges from small humanities corpora to decades of polar satellite telemetry. The team runs enterprise storage frames on premises today. This session translates that world into Azure at the architectural level: which account type, performance tier, and namespace fit which workload; how the Hot, Cool, Cold, and Archive tiers and lifecycle policies form the cost engine; how Azure Files, Azure NetApp Files, and Azure Managed Lustre map to NAS and HPC scratch; and where the scale targets sit so that a portfolio beyond one account is sharded on purpose.

Several decisions in this session are hard or impossible to reverse once data lands: account kind and performance tier, hierarchical namespace, redundancy (which is account-wide), and the account name itself. The checkpoint, the exercise, and the discussion prompts all return to those decisions.

1.2 Have ready

- The deck's Block 1 slides on account types, redundancy, and the hierarchical namespace trade-offs (the feature support matrix slide).
- Your own on-premises inventory in mind: which frames, which tiers, which NAS exports, and which HPC scratch the three data classes in the exercise live on today.
- A copy of the Research Storage Inventory workbook open if you have a laptop; the Summary sheet is the starting point for the exercise's cost line.

1.3 Materials you will receive

- Session 1 deck; demo script `S01_Demo_Script_Tiering_Cost_Model`
- `UMN_Research_Storage_Inventory.xlsx` (12,240 synthetic datasets, 3.0 PB) and `UMN_Storage_Cost_Model_Workbook.xlsx`
- Azure Storage decision matrix reference card (in the Reference Card Pack)

- This workbook (participants) and the facilitator guide (instructor)

2. Block 1 checkpoint: three questions before data lands

At the end of Block 1 the facilitator pauses on three questions. Answer them for one real dataset you are responsible for today (name it in your own words; no identifiers needed).

Question 1

Performance: does this data need premium latency, or is standard general-purpose v2 enough? What is the evidence from how it is accessed today?

Question 2

Namespace: will anyone run analytics on it or need directory-level permissions? What does enabling hierarchical namespace cost you in Blob features?

Question 3

Redundancy: what redundancy does the data's value justify, and does that force a separate account?

3. Design exercise: mapping three university data classes to Azure storage

In groups of three or four, take three representative data classes and produce a storage design for each: account type and performance tier, namespace, default and target tiers with the lifecycle rules that move data between them, redundancy, the tags that will drive lifecycle and chargeback, and a cost line. Use the Research Storage Inventory and the cost model workbook for capacity and illustrative prices.

Class A, polar satellite archives. About 1.6 PB across 2,100 datasets, objects of 100 MB to 4 GB, rarely read after the first year, irreplaceable, retention Permanent or Century, some datasets under legal hold. Today: two tape libraries and a capacity NAS.

Class B, humanities text corpus with active researchers. About 90 TB across 1,200 datasets, millions of small objects (50 KB to 3 MB), a third of it read every month by a dozen researchers, retention Reference (10 years), public or internal sensitivity. Today: the research NAS.

Class C, enterprise application data. About 40 TB of application files, exports, and attachments for the ERP and identity platforms, frequent access, business-critical, retention Project (7 years), confidential. Today: Tier 1 flash on the enterprise frame.

3.1 Steps

1. Read the three classes and assign one group member to own each class; the fourth (or the facilitator) owns the cost line.
2. For each class complete Worksheet 1: account and namespace decisions, with the reason for each.
3. Complete Worksheet 2: tiers, lifecycle rules, and tags, including the thresholds and the condition type (last access, modification, or creation).
4. Complete Worksheet 3: redundancy, sharding, and the cost line (monthly, using the cost model's illustrative prices, with the all-Hot number beside it).
5. Prepare a five-minute presentation: one slide or whiteboard per class, then the cost line and the two decisions you found hardest.
6. Present; the facilitator critiques against the five Well-Architected Framework lenses; capture what you would change.

3.2 Worksheet: account and namespace decisions

Decision	Class A: satellite archives	Class B: text corpus	Class C: enterprise application
Account kind and performance tier			
Hierarchical namespace (yes or no) and why			
Default access tier			
Features that must be available (versioning, change feed, WORM, tags)			
Naming convention (account, container)			

3.3 Worksheet: tiers, lifecycle rules, and tags

Decision	Class A	Class B	Class C
Target tier(s) and why			

Decision	Class A	Class B	Class C
Lifecycle rule: filter (tag or prefix), condition type, thresholds			
Small-object handling			
Tags applied at ingest			
Smart tier: use it or not, and why			

3.4 Worksheet: redundancy, sharding, and cost

Decision	Class A	Class B	Class C
Redundancy and justification			
Sharding: which accounts, by which pattern			

Decision	Class A	Class B	Class C
Security baseline deviations (if any)			
Monthly cost line (proposed)			
Monthly cost if everything were Hot			

3.5 What you present

- One design per class: account, namespace, tiers and rules, redundancy, tags, cost line.
- The two decisions your group found hardest and how you resolved them.
- One thing you would need to confirm with the data stewards before committing.

3.6 How your design will be critiqued

Lens	Meets	Exceeds
Reliability	Redundancy stated per class with a reason tied to the data's replaceability	Separates durability from recoverability and names the backup or versioning that covers deletion
Security	Security baseline applied (HTTPS, TLS 1.2, no anonymous access, shared key off, private endpoints noted)	Private endpoint and ABAC noted for confidential data; tags treated as governed metadata
Cost optimization	Tier and lifecycle thresholds stated, cost line produced with the all-Hot comparison	Early deletion exposure, small-object handling, and reserved capacity eligibility considered
Operational excellence	Naming convention and tagging standard stated; deployment described as template-driven	Blob inventory or cost exports named as the feedback loop for threshold tuning
Performance efficiency	Account type matched to access pattern; scale targets acknowledged	Partitioning or sharding by request profile explained; hot-partition naming avoided

4. Discussion prompts

Work through these in your group or with a neighbor during the time the facilitator calls out. Write enough that you can defend the position in the room.

Prompt 1

Which of today's on-premises tiers map cleanly to Azure, and which do not map at all?

Think about Tier 1 flash, hybrid, capacity NAS, HPC scratch, and tape.

Prompt 2

The scale targets say 5 PiB and 40,000 requests per second per account by default. For a 3 PB portfolio, when would you still shard, and by which pattern?

Patterns: by data domain, by protection class, by request profile, catalog and template.

Prompt 3

A faculty member asks why their newly uploaded files cost more to delete than to keep. What happened, and what do you change?

Early deletion charges and auto-tiering.

5. Knowledge check

Six questions, one best answer each. Mark your answer; the facilitator reviews them at the end of the session.

1. Which standard storage account type supports all four blob access tiers and smart tier?

- A. Premium block blob
- B. Standard general-purpose v2
- C. Standard general-purpose v1
- D. FileStorage

2. Which feature remains available on a hierarchical namespace account?

- A. Blob versioning
- B. Object replication
- C. Container-level WORM
- D. Point-in-time restore

3. What is the minimum retention period for the Cold tier before early deletion charges apply?

- A. 30 days
- B. 60 days
- C. 90 days
- D. 180 days

4. A 6 GB archived blob rehydrated at Standard priority can take up to how long?

- A. 1 hour
- B. 4 hours
- C. 15 hours

- D. 48 hours

5. Which statement about smart tier is correct?

- A. It moves inactive blobs to Archive after 180 days
- B. It requires a zone-redundant GPv2 account and never uses Archive
- C. It is covered by reserved capacity
- D. It works on premium block blob accounts

6. Why can a lifecycle rule on an archival account with WORM policies not use the last accessed condition?

- A. Lifecycle rules do not support last access
- B. Last access tracking cannot be enabled on an account with immutable storage
- C. Archive tier does not record access times
- D. Tags cannot be combined with last access

6. Takeaways and the capstone

- Account type, namespace, and redundancy are architecture decisions that are hard to reverse; tiers and lifecycle rules are the cost engine you tune continuously.
- Target tags, not folder paths, and model the money before enabling a rule; review affected datasets by name.
- Shard accounts by data domain, protection class, and request profile, and deploy every account from the same template.
- Azure Files, Azure NetApp Files, and Azure Managed Lustre each have one job; Blob with tiering holds bulk research data.

> **Capstone input from this session**
>
> Tagged, lifecycle-managed research storage with reserved capacity.
>
> Next: Session 2, long-term archival, data movement at scale, and hybrid identity, builds on today's tiers and lifecycle policies.

7. Notes

Your notes

